

UNIVERSITATEA TEHNICĂ A MOLDOVEI

Технический университет Молдовы

Факультет вычислительной техники,
информатики и микроэлектроники

Департамент программной инженерии и автоматике

Программирование мобильных приложений

Programarea aplicațiilor mobile

Отчёт по лабораторной работе № 1

*Среда разработки, язык Dart
и выбор темы проекта*

Выполнил: Намуев Oleg

Группа: CR-232

Проверил: Leah A.

Учебный год 2026–2027

Кишинёв 2026

1 Среда разработки

Работа выполнена на macOS с Flutter 3.47.0 и Dart 3.13.0. Основная платформа проекта — iOS. Используются Xcode 26.6, CocoaPods 1.17.0 и iPhone 17 Pro Simulator. Ниже приведён полный фактический вывод flutter doctor, полученный после настройки среды.

```
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.47.0, on macOS 26.6.2 25G83 darwin-arm64, locale en-US)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
[✓] Xcode - develop for iOS and macOS (Xcode 26.6)
[✓] Chrome - develop for the web
[✓] Connected device (3 available)
[✓] Network resources

• No issues found!
```

Все разделы flutter doctor прошли проверку. Для демонстрации используется iOS Simulator, в соответствии с платформой RunBaza.

2 Устройства и запуск demo

```
Found 3 connected devices:
  iPhone 17 Pro (mobile) • 4528ABEC-4EB2-47AE-AC2A-8A5DA1F029D0 • ios
  com.apple.CoreSimulator.SimRuntime.iOS-26-5 (simulator)
  macOS (desktop)      • macos
  darwin-arm64          • darwin-arm64 • macOS 26.6.2 25G83
  Chrome (web)          • chrome
  153.0.8010.53         • web-javascript • Google Chrome

Checking for wireless devices...

No wireless devices were found.

Run "flutter emulators" to list and start any available device emulators.

If you expected another device to be detected, please run "flutter doctor" to diagnose potential issues.
You may also try increasing the time to wait for connected devices with the "--device-timeout" flag.
Visit https://flutter.dev/setup/ for troubleshooting tips.
```

Приложение hello_pam создано командой flutter create и запущено на iPhone 17 Pro (iOS 26.5). Нажатие кнопки «+» изменяет счётчик с 0 на 1. Стандартный widget-тест счётчика также проходит.

```
flutter create --platforms=ios hello_pam
cd hello_pam
flutter run -d 4528ABEC-4EB2-47AE-AC2A-8A5DA1F029D0
```

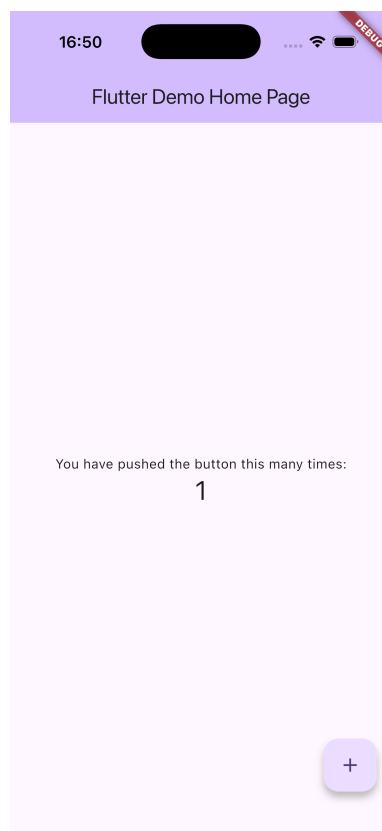


Рисунок 1. Фактический экран iPhone Simulator после нажатия «+».

E1 Переменные и null safety

String? допускает null. int.tryParse возвращает null для некорректной строки, а ?? подставляет fallback. Оператор принудительного раскрытия null не используется.

Исходный код

```
int parseOrDefault(String? input, int fallback) =>
    int.tryParse(input ?? '') ?? fallback;
```

Вызовы из main

```
print(parseOrDefault('42', 0));
print(parseOrDefault('abc', 0));
print(parseOrDefault(null, 7));
print(parseOrDefault('7.5', -1));
```

Результат запуска

```
42
0
7
-1
```

Е2 Коллекции и функциональные методы

where оставляет оценки не ниже 5. fold суммирует их; деление на length даёт среднее. При пустой выборке возвращается 0.0; циклов for и while в функции нет.

Исходный код

```
double promotedAverage(List<int> grades) {  
    final promoted = grades.where((grade) => grade >= 5);  
    return promoted.isEmpty  
        ? 0.0  
        : promoted.fold<int>(0, (sum, grade) => sum + grade) / promoted.length;  
}
```

Вызовы из main

```
print(promotedAverage([9, 4, 7, 10, 3, 5]));  
print(promotedAverage([10, 10, 9]));  
print(promotedAverage([3, 4, 2]));
```

Результат запуска

```
7.75  
9.666666666666666  
0.0
```

Е3 Именованные параметры

currency и decimals передаются по имени. Значения по умолчанию — MDL и 2. toStringAsFixed форматирует число с указанным количеством знаков.

Исходный код

```
String formatPrice(  
    double amount, {  
        String currency = 'MDL',  
        int decimals = 2,  
    }) => '${amount.toStringAsFixed(decimals)} $currency';
```

Вызовы из main

```
print(formatPrice(12.5));  
print(formatPrice(12.5, currency: 'MDL'));  
print(formatPrice(99.999, currency: 'EUR'));  
print(formatPrice(7.5, currency: 'USD', decimals: 1));
```

Результат запуска

```
12.50 MDL  
12.50 MDL  
100.00 EUR  
7.5 USD
```

Е4 Класс Student и равенство

Оба поля final. Именованный конструктор guest перенаправляет вызов в основной. Равенство сравнивает значения name и group; Object.hash использует те же поля, поэтому Set устраняет дубликаты.

Исходный код

```
class Student {
    final String name;
    final String group;

    const Student(this.name, this.group);
    const Student.guest() : this('Guest', '-');

    @override
    String toString() => 'Student($name, $group)';

    @override
    bool operator ==(Object other) =>
        identical(this, other) ||
        other is Student && other.name == name && other.group == group;

    @override
    int get hashCode => Object.hash(name, group);
}
```

Вызовы из main

```
print(Student('Ana Rusu', 'TI-231'));
print(Student.guest());
print(Student('Ana Rusu', 'TI-231') == Student('Ana Rusu', 'TI-231'));
print({Student('Ana Rusu', 'TI-231'), Student('Ana Rusu', 'TI-231')}.length);
```

Результат запуска

```
Student(Ana Rusu, TI-231)
Student(Guest, -)
true
1
```

E5 Mixin Loggable

with добавляет общий метод log двум независимым классам. runtimeType выводит имя фактического класса объекта: CartService или AuthService.

Исходный код

```
mixin Loggable {  
  void log(String msg) => print('[$runtimeType] $msg');  
}  
  
class CartService with Loggable {  
  void addItem(String item) => log('товар добавлен: $item');  
}  
  
class AuthService with Loggable {  
  void login(String username) => log('пользователь вошёл: $username');  
}
```

Вызовы из main

```
CartService().addItem('Paracetamol');  
AuthService().login('ana.rusu');
```

Результат запуска

```
[CartService] товар добавлен: Paracetamol  
[AuthService] пользователь вошёл: ana.rusu
```

Е6 Асинхронная загрузка профиля

`fetchProfile` возвращает `Future<String>`. `await` ожидает завершение `Future.delayed` в течение двух секунд, после чего `main` печатает профиль и «Готово.». Цикл событий во время ожидания остаётся свободным.

Исходный код

```
Future<String> fetchProfile(String username) async {  
  await Future<void>.delayed(const Duration(seconds: 2));  
  return 'Профиль: $username, группа TI-231, решено упражнений: 42';  
}
```

Вызовы из main

```
Future<void> main() async {  
  print('Загружается профиль пользователя ana.rusu...');  
  final profile = await fetchProfile('ana.rusu');  
  print(profile);  
  print('Готово.');
```

Результат запуска

```
Загружается профиль пользователя ana.rusu...  
Профиль: ana.rusu, группа TI-231, решено упражнений: 42  
Готово.
```

Дополнительная проверка `check.dart`: 17 проверок завершились успешно. Проверены примеры, пустой список, различающиеся студенты, `hashCode`, длительность ожидания и выполнение другой асинхронной задачи во время ожидания.

Приложение Мини спецификация RunBaza

RunBaza — календарь забегов и совместные поездки

Собственная тема семестрового проекта. Основание: README репозитория HamuevOleg/PAM.

Закрепление собственной темы подтверждает преподаватель на защите L1.

Целевая аудитория

Бегуны-любители из Молдовы и соседних городов Румынии. Приложение объединяет информацию о стартах и помогает найти попутчиков для поездки на забег за пределами Кишинёва.

Экраны

1. Home / Board — ближайший старт, количество забегов, свободные места, максимальная дистанция, дни до старта.
2. Race Index — забеги по сезонам; поиск и фильтры по месяцу, дистанции, городу, статусу и типу.
3. Race Details — время, место встречи, дистанции, взнос, скидки, заметки организатора и ссылка регистрации.
4. Transfer — предложения водителей и заявки пассажиров для выбранного забега; время, место и свободные места.
5. My Plan — избранные забеги и личные напоминания.
6. Settings — тема, уведомления и домашний город.
7. Login — вход или регистрация; обязательное дополнение по условию L2.

Сущности

Race: id, code, title, date, time, city, venue, distanceFromChisinau, distances, type, status, fee, currency, discount, organizerNote, registrationUrl.

TransferOffer: id, raceId, driverName, departureTime, pickupPoint, freeSeats, contribution. SeatRequest: id, raceId, passengerName, seats, pickupPoint.

User: id, name, email, homeCity. PlanEntry: id, userId, raceId, reminderDays. AppSettings: themeMode, notificationsEnabled, homeCity.

Этапы L2–L6

L2 — статические экраны Material 3 и mock-данные. L3 — маршруты и валидация форм. L4 — состояние, фильтры, избранное и локальное хранение. L5 — удалённые данные и аутентификация. L6 — напоминания, проверка и завершение приложения.

Почему эта тема

Тема позволяет объединить календарь спортивных событий и практическую задачу поездки на старт.

Концепция табло отправлений делает статусы забегов наглядными; на её основе удобно последовательно изучать интерфейс, состояние, хранение данных и уведомления.